Making an ECC-based embedded SPDM device quantum-resistant

Final report - PQC Hackathon Challenge, SEMICON 2026 (Infineon track)

Boot-time protection (Edge Protect Bootloader, ML-DSA-87) and real-time protection (SPDM 1.4, ML-DSA-87 / ML-KEM-1024) on the PSOC™ Control C3M6 evaluation kit (KIT_PSC3M6_EVAL, PSC3M6GES3A)

Item	Value
Date	12 November 2026
Target profile	CNSA 2.0: ML-DSA-87 (FIPS 204), ML-KEM-1024 (FIPS 203), SHA-384/512, AES-256 - design constraint, no certification claim
Hardware	KIT_PSC3M6_EVAL (Cortex-M33 + TrustZone, 512 KB flash, 128 KB SRAM), lifecycle DEVELOPMENT
Tooling	ModusToolbox 3.9, GCC 14.2.1, Edge Protect Tools, Infineon EPBL (CE43373), CE243041, Infineon SPDM CE (libspdm, wolfCrypt, mbedTLS)
Code and evidence	https://github.com/kiryl-web/hackathon_semi_munich (branch pqc-task1-task2)
Detailed reports	results/Task1_ECC_Baseline_Report.pdf, Task2_PQC_Boot_Report.pdf, Task3_PQC_SPDM_Report.pdf, Task4_Lifecycle_Report.pdf, Task5_Presentation.pdf

All results in this report were measured on the evaluation board. Every number links to a log or run directory in the repository (section 12). Where we did not fully meet the brief, we say so (section 9).

1. Executive summary

We started from Infineon's ECC-based SPDM code example and migrated both protection planes of the device to CNSA 2.0 post-quantum algorithms on real PSoC Control C3 hardware. We did not use a hybrid design, because the brief excludes it:

- **Boot-time plane:** the Edge Protect Bootloader (EPBL) verifies the application with **ML-DSA-87 (HashML-DSA, SHA-512)** before it runs, enforces the image version against rollback, and rejects classical, weaker-PQC and malformed images. No ECC fallback exists.
- **Real-time plane:** the SPDM responder runs **SPDM 1.4 with ML-DSA-87 device identity and ML-KEM-1024 key establishment**, SHA-384 and AES-256-GCM. It offers no classical algorithm at all, so no downgrade is possible.
- **One lifecycle:** the real SPDM responder now boots behind EPBL. Its signed SPDM measurements report the EPBL digest, the digest of the exact image EPBL verified, and the version. A host script recomputes these from the release files and they matched after every boot. Firmware updates keep the provisioned identity, and the host reconnects without re-provisioning.

Result	ECC baseline (Task 1)	PQC migration (Tasks 2-4)
Boot tests on hardware	8/8 as expected (1 boot, 6 rejections, 1 update)	4/4 positive, 11/11 negative rejected (CE243041 app); 18/18 as expected with the SPDM responder behind EPBL
SPDM tests on hardware	full suite PASS; 6/6 tamper cases + wrong root rejected	full suite PASS; 12/12 tamper cases rejected; reconnect after tamper and after update PASS
Credential lifecycle	ECC identity provisioned on-chip	ML-DSA-87 identity generated on-chip; rotation PASS; 5 invalid-credential cases rejected
Boot signature verification	ECDSA-P384: ~170 ms	**ML-DSA-87: ~61 ms** (2.8x faster; P-384 is software-only on this part)
Session key establishment (device)	ECDHE-P384: 163 ms	**ML-KEM-1024 encaps: 11 ms** (15x faster)
Main cost	-	**RAM and bytes**: +38 KB SPDM buffers, peak handshake RAM ~108 of 126 KB; chain 1.2 -> 15.2 KB; 11x session traffic
Remaining classical link	-	ROM_BOOT/FLASH_BOOT -> EPBL (ECDSA in immutable ROM; not enabled on the shared board)

Main finding. On a Cortex-M33 the post-quantum algorithms are not the bottleneck. Verification and key establishment are faster than software P-384, and flash use is flat. The cost is the size of the PQC objects: certificate chains, signatures and transcripts grow by about 10x. That shows up as RAM, as a 99 %-full credential slot, and as transport time over a 127-byte-per-transaction I2C tunnel. This is the budget a product team must plan for.

2. Compliance driver and selected algorithms

CNSA 2.0 is the design constraint. We did not take the commercial default parameter sets (ML-DSA-65, ML-KEM-768) on trust, because CNSA 2.0 requires NIST security category 5 for all uses. The selected profile and its mapping from the original ECC flow:

Function	Original ECC flow	CNSA 2.0 requirement	Selected (implemented)
Firmware signature (EPBL -> app)	ECDSA P-384 / SHA-384	ML-DSA-87, or LMS/XMSS (SP 800-208) for firmware	**ML-DSA-87**, HashML-DSA over SHA-512
Image hash	SHA-384	SHA-384 or SHA-512	**SHA-512** (SHA-256 pre-hash rejected, test N9)
SPDM device authentication	ECDSA P-384 certificates	ML-DSA-87	**ML-DSA-87** chain (root -> leaf), SPDM 1.4
SPDM key establishment	ECDHE P-384	ML-KEM-1024	**ML-KEM-1024**, responder encapsulates in the secure world
Transcript hash / KDF	SHA-384 / HKDF	SHA-384 or SHA-512	SHA-384 / HKDF-SHA-384
Session encryption	AES-256-GCM	AES-256	AES-256-GCM (unchanged)
BootROM -> EPBL	ECDSA (OEM key)	-	remains classical (immutable ROM), see section 9

- **Why ML-DSA-87 and not LMS for firmware:** both are allowed and both are supported by EPBL. ML-DSA-87 is stateless, so a build server does not have to track one-time-key state. It is also the same family as the SPDM identity, so one PQC algorithm covers both planes. LMS would give a smaller verifier and can be dropped in instead.
- **Why SHA-512 pre-hash:** the documented EPBL default for ML-DSA is SHA-256. That gives 128-bit collision resistance, below category 5, and is not a CNSA 2.0 hash. We also found that the EPBL and the CE243041 signer defaulted to *different* hashes, which produces images that never boot. We added an explicit `IMAGE_SHA` option to the signer.

- **Enforcement:** ML-DSA-65 images, ML-DSA-65 roots, ML-DSA-65 key generation and ECDSA-signed images are all rejected on hardware (boot N8/N11, SPDM A2/A3, provisioning V3/V4).

3. Starting point: ECC architecture and trust model (Task 1)

3.1 Boot chain as found on the board

```
Reset
 +- ROM_BOOT   (immutable mask ROM, Infineon keys)          classical; cannot verify ML-DSA
     +- FLASH_BOOT (Infineon, applies lifecycle + boot policy)  policy SIMPLE_APP: launches EPBL WITHOUT authentication
         |                                                       (SECURE_APP would verify EPBL with an ECDSA OEM key)
         +- EPBL @0x32000000 (MCUboot-based, first OEM code)    verifies the next image: ECDSA-P384 (baseline)
             +- application in primary slot                     CE243041 demo app (Tasks 1-2) / SPDM responder (Task 4)
```

- Lifecycle state read from the device: **DEVELOPMENT** (debug port open, PAS not frozen), as the brief allows for an open hackathon.
- Image format: MCUboot, 0x400-byte header (version), payload, TLV area (image hash, key hash, signature, protected TLVs). Overwrite-mode primary/upgrade slots with software downgrade prevention (`sw_downgrade_prev`).
- OEM root-of-trust key: none provisioned (no ownership transfer). Enabling FLASH_BOOT authentication of EPBL needs an Infineon-signed OEM development certificate (AN243370 / AN243376) that we did not have. It was therefore not enabled on the shared board.

3.2 Baseline results

Plane	Test	Result
Boot	authorized v1.0.0 (ECDSA-P384)	BOOTED
Boot	1 payload byte flipped / header version byte / protected TLV / signature byte	all REJECTED (hash or signature check)
Boot	rogue ECDSA key; rollback v0.9.0 in upgrade slot	REJECTED; rollback slot erased (flash readback before/after)
Boot	valid update v2.0.0+1	INSTALLED, re-verified, runs
SPDM	connect, digests, get-cert (1,218 B chain), challenge, measurements, ECDHE session, 3 encrypted exchanges	PASS
SPDM	wrong trust anchor; 6 fault-injection images (cert, signature, transcript, AEAD tags)	all REJECTED

Evidence: results/boot_ECDSA-384_*.log, results/boot_evidence/, results/task1_P2_20261111_1215/, results/task3_EDGE_P2FI_20261111_1753/ (baseline tamper campaign). Measurements are in section 8.

4. Boot-time protection with ML-DSA-87 (Task 2)

4.1 Design

- **BootROM vs EPBL:** we do **not** assume that BootROM verifies ML-DSA (AN243370 sec. 1.3: BootROM secure boot is ECDSA). The PQC check is done by EPBL, the first OEM stage, on the primary slot at every boot. It also checks the upgrade slot before installing (signature + version) and the primary slot again after copying.
- **Validation key:** the ML-DSA-87 public key (2,592 B) is compiled into the EPBL flash image (`generated/ml_dsa_sign_key.c`). The `sflash_keys` option is not available for ML-DSA. The key's integrity therefore depends on EPBL integrity, which in production comes from BootROM authentication of EPBL plus a closed debug port.
- **Metadata:** SHA-512 image hash TLV (64 B), key-hash TLV (64 B), ML-DSA-87 signature TLV (4,627 B). The TLV area grows from 215 B to 4,771 B. The header version is covered by the hash, so editing it breaks the signature.
- **Configuration:** `epbl/bl/platforms/PSC3_M6/feature_config_ML-DSA-87.json` (PQC) and `feature_config_ECDSA-384_baseline.json` (baseline).

4.2 Test matrix on hardware (CE243041 application)

#	Case	Result	EPBL evidence
P1	authorized v1.0.0, ML-DSA-87 + SHA-512	BOOTED	validate 0x742e
N1	payload byte flipped	REJECTED	IMAGE_TLV_SHA invalid
N2	MCUboot header version byte modified	REJECTED	IMAGE_TLV_SHA invalid
N3	protected TLV byte modified	REJECTED	IMAGE_TLV_SHA invalid
N4	key-hash TLV modified	REJECTED	0x2e, no matching key
N5	byte inside the ML-DSA-87 signature	REJECTED	hash OK, ML-DSA verify fails

#	Case	Result	EPBL evidence
N6	malformed: signature TLV length 4627 -> 4611	REJECTED	0xffffffff
N7	complete image signed with a rogue ML-DSA-87 key	REJECTED	0x2e
N8	ECDSA-P384-signed image (classical downgrade)	REJECTED	no ECC fallback
N9	ML-DSA-87 with SHA-256 pre-hash (non-compliant)	REJECTED	0xffffffff
N11	ML-DSA-65 image (unsupported parameter set)	REJECTED	0x2e
N10	rollback: genuine v0.9.0 in upgrade slot	REJECTED	insufficient version; slot erased
P2-P4	restore v1.0; valid update v2.0.0+1; reboot after update	BOOTED / INSTALLED / BOOTED	readback: primary version 2
L1	genuine v0.9 written directly to the primary slot by debugger	BOOTS (limitation)	SW downgrade check guards only the update path

Evidence: results/task2_boot/ (boot logs, tampered images, readbacks, summary_ML-DSA-87.txt), runner tools/task2_boot_tests.ps1. L1 is closed in production by the eFuse anti-rollback counter (`hw_rollback_prot`, irreversible) plus a closed DAP.

5. Real-time protection: SPDM 1.4 with ML-DSA-87 / ML-KEM-1024 (Task 3)

5.1 How the host validates the device

The requester holds the ML-DSA-87 root certificate as its trust anchor. It fetches the 15,187-byte chain (root 7,578 B -> leaf 7,557 B) through GET_DIGESTS/GET_CERTIFICATE and verifies every ML-DSA-87 signature up to the pinned root. It then sends CHALLENGE: the device signs the transcript with the leaf key (FIPS 204 context = SPDM operation string). Only after that does the requester trust measurements or open a session. NEGOTIATE_ALGORITHMS offers BaseAsym = ML-DSA-87 and KEM = ML-KEM-1024 only. Because the negotiation is part of the signed transcript, a man-in-the-middle cannot strip algorithms.

5.2 How the session secret is established and derived

KEY_EXCHANGE (1,638 B) carries a fresh ML-KEM-1024 encapsulation key from the requester. The secure world encapsulates, producing a 32-byte shared secret and a 1,568-byte ciphertext. KEY_EXCHANGE_RSP (6,305 B) returns the ciphertext and the ML-DSA-87 signature over the transcript. Both sides run the SPDM key schedule (HKDF-SHA-384 over the shared secret and transcript hash), producing handshake secrets, finished keys (confirmed by FINISH) and AES-256-GCM data keys. A fresh KEM key pair per session gives forward secrecy.

5.3 How credentials and trust anchors are provisioned and updated

- The provisioning firmware (built for the `ml-dsa-87` suite) generates the ML-DSA-87 key pair **on-chip in the secure world**. The private key never leaves the device. It emits a CSR, our ML-DSA-87 test CA signs it, and the chain is written to the PAS (Provisioning Asset Store, 24 KB secure flash: key slot 8 KB, certificate slot 15 KB). Script: tools/task3_provision_p4.ps1 (41 s). Proof-of-possession is checked with `prove-cert`.
- Trust anchors live on the host (pinned root). Rotation = new on-chip key + CSR + new leaf under the same root. This was shown in Task 4, and the SPDM session still passed afterwards.
- Production steps not executed on the shared board: `--freeze` the PAS and close the debug port. Until then, a key can be regenerated (finding V5b, section 6.3).

5.4 How large PQC objects fit in the MCU memory model

All PQC buffers are static and size-checked. No allocation grows with message size. libspdm max message 7,168 B (ECC 2 KB); non-secure libspdm scratch 35,843 B + chain copy 16,384 B + context 3,328 B. Secure side: wolfCrypt ML-DSA key object 7,912 B and hash pool 9,280 B. Peak secure heap 27.7 KB of 31 KB. Messages larger than one transport unit are split with SPDM CHUNK_SEND/CHUNK_GET over a 127-byte DFU/I2C transaction. The tightest resource is the PAS certificate slot (15,187 of 15,360 B, 99 %): there is no room for an intermediate CA. Chains larger than the slot are rejected by the provisioning firmware (V6).

5.5 Tests on hardware

#	Tampering (PQC profile)	Detected at
A0/A3	requester trusts a classical ECDSA P-384 root	certificate validation (EAUTH)
A1	rogue ML-DSA-87 root with the same subject	certificate validation (EAUTH)

#	Tampering (PQC profile)	Detected at
A2	ML-DSA-65 root (non-CNSA parameter set)	certificate validation (EAUTH)
A4 / A5	1 byte flipped in root public key / truncated root DER	certificate validation (EAUTH)
B1	leaf certificate CN modified on the device	get_certificate (EAUTH)
B2	ML-DSA-87 signature byte flipped	CHALLENGE verification (EAUTH)
B3	valid signature over a modified transcript	session start (ESESSION)
B4	ML-KEM ciphertext modified before signing	key confirmation fails (implicit rejection)
B5 / B6	MITM: AEAD tag of FINISH_RSP / of application data	AES-GCM tag check (ESESSION)

Positive: connect, digests, get-cert, challenge, measurements, ML-KEM session and encrypted exchanges PASS. Reconnect after the tamper images PASS. The B tests use a test-only fault-injection build (`SPDM_FAULT_INJECT=1..6`, compiled out by default, banner `[FI]`). One of the first three exchanges aborted once at the last KEY_EXCHANGE_RSP chunk: the responder lost state and the connection **failed closed**. It did not reproduce in 9 later full handshakes, and the root cause is unknown (section 9). Offline test suites: tools/tests (58 checks OK). Evidence: results/task3_P4_20261111_1449/, task3_NEG_P4_20261111_1533/, task3_EDGE_*.

6. Lifecycle connection: boot and SPDM as one device (Task 4)

In Task 3 the responder ran without EPBL, so the device answering SPDM was not verified at boot. In Task 4 we put the **real SPDM responder behind EPBL**. This needed a new BSP variant (`APP_KIT_PSC3M6_EVAL_EPBL`), regenerated TrustZone/MPC regions, and a signing tool (tools/resp_epbl_sign.py) that merges the secure and non-secure images into one ML-DSA-87-signed MCUboot image.

Area	Range	Content
EPBL	0x32000000 - 0x3201B000 (108 KB)	bootloader, ML-DSA-87 validation key embedded
Primary slot	0x3201B000 - 0x32048000 (180 KB)	header 0x400 + responder S + NS + TLVs
PAS	0x32048000 - 0x3204E000 (24 KB)	SPDM identity (ML-DSA-87 key + chain), untouched by updates
Upgrade slot	0x3204E000 - 0x3207B000 (180 KB)	candidate image (overwrite mode)

6.1 Binding attestation to verified boot

The example's placeholder measurements were replaced in the secure world with: block 1 = SHA-384 of the EPBL area, block 2 = SHA-384 of exactly the bytes EPBL hashed before verifying the ML-DSA-87 signature, block 3 = security version (MCUboot version), block 4 = version string. They are signed with the ML-DSA-87 identity in GET_MEASUREMENTS. tools/task4_verify_meas.py recomputes blocks 1-3 from the release files. They **matched after every boot**, so the host has quantum-safe signed evidence of which verified image is running.

6.2 Firmware update path (18/18 as expected)

Step	Result	Evidence
T0 boot v1.0.0 -> measure / exchange	BOOTED, PASS / PASS	ML-DSA-87 verify 80 ms, reset->responder 157 ms, MATCH
T1 valid update v1.1.0 -> reconnect, measure / exchange	INSTALLED, PASS / PASS	same identity, SVN 1.1.0, new digest, MATCH; reset->app 4.95 s
T2 rollback to genuine v1.0.0	REJECTED	insufficient version in secondary slot
T3 corrupt payload / T4 modified header	REJECTED	SHA-512 mismatch (0xffffffff)
T5 image signed with rogue ML-DSA-87 key	REJECTED	0x2e, no matching key hash
T6 measure after rejections	PASS	still 1.1.0, MATCH: no rejected image ran
T7 tampered primary image	NO BOOT	primary not valid, handle_error; SPDM unreachable
T8 recovery update v2.0.0 -> measure / exchange	INSTALLED, PASS / PASS	SVN 2.0.0, MATCH

6.3 Credential update path (PAS)

- **Accepted:** rotation to a new on-chip ML-DSA-87 key + CA-signed leaf (same root), followed by encrypted SPDM exchanges (PASS).
- **Rejected:** ML-DSA-65 key generation (PROV_ERR_ALG_UNSUPPORTED), ECDSA key generation in the PQC build (same), chain larger than the 15,360 B slot (PROV_ERR_LENGTH), chain with SHA-256 RootHash vs negotiated SHA-384 (host EAUTH).
- **Leaf certificate for a foreign key:** stored (the device does not parse X.509), but proof-of-possession fails and so would SPDM CHALLENGE. The provisioning process must therefore gate on `prove-cert`.

- **Finding V5b:** key generation on an occupied slot is accepted while the PAS is not frozen (by design; a frozen device returns PROV_ERR_FROZEN). Credential immutability in the field depends on `--freeze`, a closed debug port, and the provisioning image never being bootable.

6.4 Where the SPDM identity is anchored

In both. The signing key is a separately provisioned credential in the PAS. It survives updates and the private key never leaves the secure world. The firmware state is anchored in verified boot through the signed measurements. Consequence: if EPBL were bypassed (possible today because BootROM does not authenticate EPBL and the debug port is open), malicious code could try to use the key through the secure-world API. It could not forge measurements without also compromising the secure firmware. BootROM authentication of EPBL plus a closed DAP closes this path. A DICE-style key derived from the measured image would bind the two cryptographically (next step). Evidence: results/task4_EPBL_SPDM_20261111_2059/, task4_EDGE_PROV_20261111_1903/, task4_EDGE_RESTORE_20261111_1912/.

7. Validation scenarios required by the brief

Required scenario	Baseline	PQC	Where
ECC baseline image verification and boot	PASS	-	Task 1, boot_ECDSA-384_v1_0.log
PQC next-image verification and boot through EPBL	-	PASS	Task 2 P1, Task 4 T0
Valid firmware / credential update + reboot	PASS	PASS	Task 2 P3/P4, Task 4 T1/T8, credential rotation
ECC baseline SPDM session	PASS	-	task1_P2_20261111_1215
PQC SPDM session (ML-KEM-1024)	-	PASS	task3_P4_*, task3_EDGE_RESTORE_*
Responder authentication after boot	PASS	PASS	challenge + measurements after EPBL boot (Task 4)
Reconnect after update	-	PASS	Task 4 T1/T8, same identity
Modified responder certificate / credential	REJECTED	REJECTED	B1, A1-A5, V3/V4/V6/V9
Invalid signature	REJECTED	REJECTED	boot N5/N7, SPDM B2
Modified MCUboot metadata / manifest	REJECTED	REJECTED	boot N2/N3/N4/N6, T4
Unauthorized rollback image	REJECTED	REJECTED	N10, T2 (update path; see L1)
Modified SPDM transcript / handshake message	REJECTED	REJECTED	B3, B4, B5, B6
Unsupported or malformed PQC key material	-	REJECTED	N9, N11, A2, A4, A5, V3
No silent fallback to classical	-	REJECTED	boot N8 (ECDSA image), SPDM A0/A3 (ECC root), no classical alg offered

Live demonstration (Task 5): tools/demo_app (full-screen app) and tools/demo/*.cmd (backup buttons). Dry run: prepare 19 s, authorized boot 7 s, rogue-key update rejected 16 s, valid update installed 20 s, rollback rejected 13 s. SPDM measurement after update PASS with 3x MATCH. One SPDM operation takes 6-10 min over the I2C tool, so it is started before the talk.

8. Resource and performance comparison

Same board and code base; Release builds for the responder. Device timings come from on-chip instrumentation (`[PERF]`, NS SysTick) and EPBL log timestamps. RAM peaks come from stack painting and the heap high-water mark.

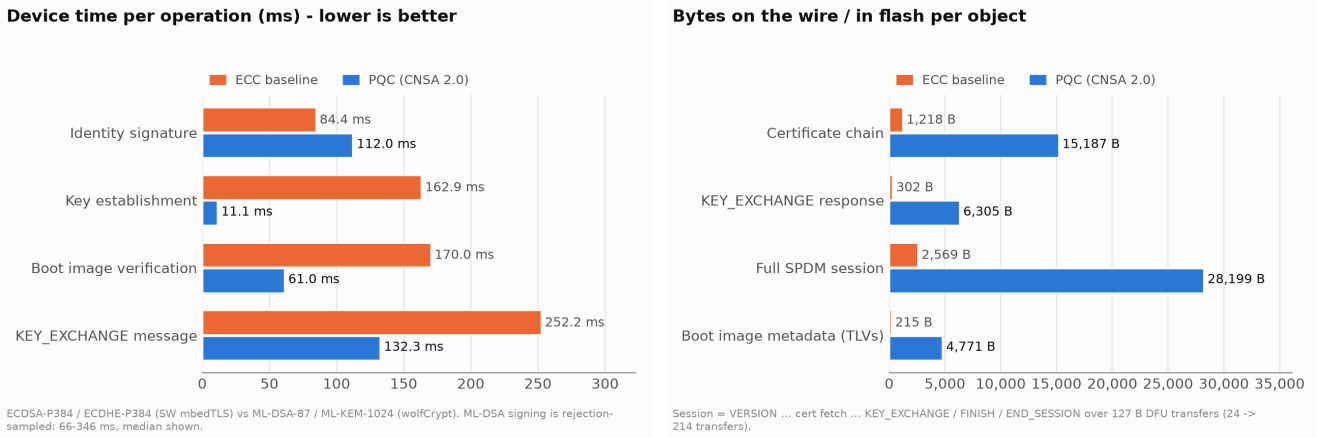
8.1 Boot-time costs (EPBL)

Metric	ECDSA-P384	ML-DSA-87 (SHA-512)	Delta
Signature verification	~170 ms	**~61 ms** (80 ms for the 180 KB responder image)	-109 ms
boot_validate_slot (hash + key hash + signature)	~179 ms	~69 ms	-110 ms
Reset -> application start	~266 ms	**~146 ms** (157 ms responder)	-120 ms
Reset -> app with update install (erase + copy + 2 verifications)	~3.80 s	~3.57 s (4.95 s, 180 KB slot)	dominated by flash erase
EPBL flash (.text + .data)	35,888 B	47,244 B (43 % of 108 KB slot)	+11,356 B
EPBL static RAM (.data + .bss)	6,948 B	19,044 B	+12,096 B (ML-DSA key object)
EPBL peak stack during verification	3,752 B	6,120 B	+2,368 B
Peak boot-time RAM	~10.7 KB	**~25.2 KB** (of 54 KB EPBL region)	+14.5 KB

Metric	ECDSA-P384	ML-DSA-87 (SHA-512)	Delta
Image metadata (TLV area) per image	215 B	4,771 B	+4,556 B
Persistent trust anchor (validation key)	120 B (P-384 DER)	2,592 B raw	+2.5 KB in EPBL flash

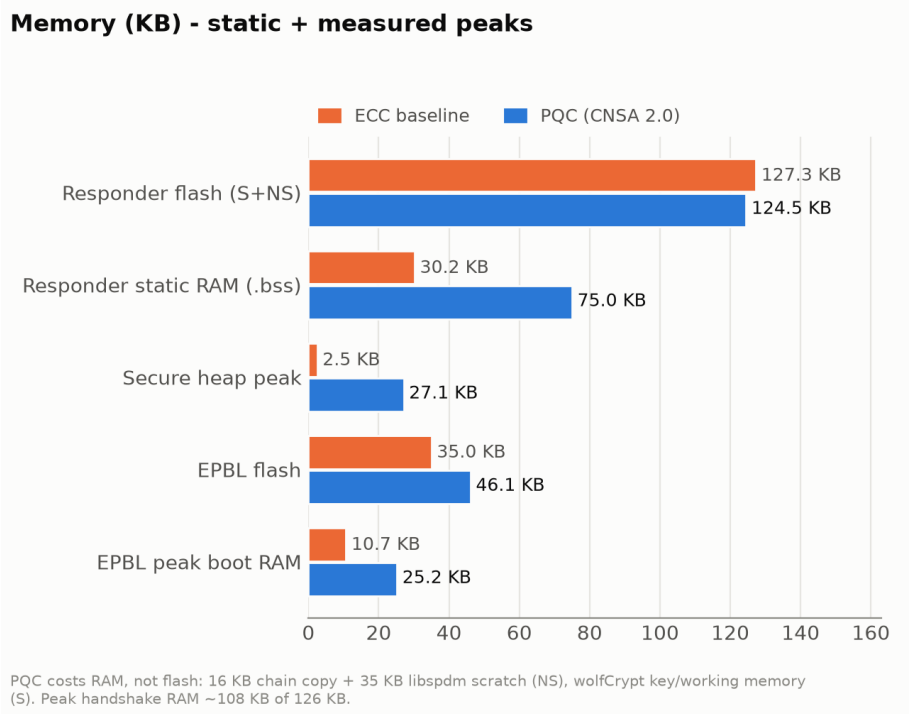
8.2 Runtime costs (SPDM responder)

Metric	P2: ECDSA/ECDHE P-384	P4: ML-DSA-87 / ML-KEM-1024	Delta
Identity signature (secure world)	84.4 ms (constant)	66-346 ms, median 112 ms (rejection sampling)	~+33 % median
Key establishment on device	163 ms (keygen + shared secret)	**11.1 ms** (encapsulation)	15x faster
KEY_EXCHANGE message (device)	252 ms	87-315 ms, median 132 ms	-48 % median
FINISH / encrypted app message	5.0 / 1.2 ms	5.9 / 1.9 ms	small
Responder flash (S + NS)	130,336 B	127,504 B	-2,832 B
Static RAM: NS .bss / S .bss	19,952 / 11,016 B	57,816 / 18,940 B	+37,864 / +7,924 B
Peak S heap / S stack	2,576 / 4,336 B	27,720 / 2,608 B	+25 KB heap
Peak handshake RAM (static + peaks)	~40 KB	**~108 KB of 126 KB**	+68 KB
Credential storage (PAS)	1,218 B chain in 15 KB slot	15,187 B chain (99 % of slot)	x12.5
Largest SPDM message	1,226 B (CERTIFICATE)	7,168 B (= max message size)	x5.8
KEY_EXCHANGE request / response	166 / 302 B	1,638 / 6,305 B	x10 / x21
Full session: DFU transfers / wire bytes	24 / 2,569 B	214 / 28,199 B	x11
get-cert: transfers / bytes	17 / 1,769 B	142 / 18,490 B	x10
Fragmentation	KEY_EXCHANGE in 2 CHUNK_SEND + 3 CHUNK_GET	chain and KEY_EXCHANGE in ~127 B chunks	-
Host wall clock per session	35-38 s	~375 s (6-11 min per op in Task 4)	transport-bound



Left: device time per operation. Right: communication volume. Charts from results/task5_charts.

Interpretation. P-384 runs in software mbedTLS on this part (CryptoLite accelerates only P-256), while wolfCrypt's ML-DSA verify and ML-KEM are NTT arithmetic plus SHAKE. PQC is therefore faster for verification and key establishment, and flash is flat or smaller. The cost is RAM (the non-secure SPDM buffers almost triple) and bytes. Host wall clock is dominated by the 127-byte DFU transaction tool (~1.4 s per transaction), not by the device, which is busy for well under 1 s per session. **Not measured:** host-side ML-DSA verification and ML-KEM decapsulation times. The requester is a prebuilt binary, and in our design the device only encapsulates.



9. Remaining classical dependencies and limitations

We do **not** claim full quantum resistance for the device. Everything we control from EPBL onward is PQC. The first link is classical by silicon design.

Item	Status today	Required before claiming full quantum resistance
ROM_BOOT / FLASH_BOOT -> EPBL	classical ECDSA; on the shared board not enabled (SIMPLE_APP), EPBL launched unauthenticated	OEM ownership transfer + SECURE_APP now (still ECDSA); a PQC-capable immutable ROM needs new silicon
EPBL -> application	**ML-DSA-87, PQC**	done
SPDM identity, signatures, key establishment	**ML-DSA-87 / ML-KEM-1024, no classical fallback**	done
Session crypto	AES-256-GCM, SHA-384	done (quantum-adequate)
EPBL validation-key rotation	EPBL not field-updatable in this map	BootROM-authenticated EPBL update path
Rollback	software downgrade prevention (update path) + version in signed measurements	eFuse anti-rollback counter + closed DAP (L1)
Credential store	not frozen; provisioning channel unauthenticated	`--freeze` after provisioning; authenticated re-provisioning
Lifecycle / debug	DEVELOPMENT, debug port open	PRODUCTION/SECURE lifecycle, DAP closed
Update delivery	debugger writes the upgrade slot	in-band update over the SPDM/MCTP transport
Transport	127 B DFU transactions over I2C	MCTP over I3C / PCIe VDM, chunk retry

Other open items, documented rather than hidden:

- BootROM authentication of EPBL was not demonstrated. It needs an Infineon EPSS-signed OEM development certificate and steps we would not run on a shared board.
- One unexplained SPDM transport fault in Task 3 (responder lost state mid KEY_EXCHANGE_RSP). It failed closed and was not reproduced in 9 later handshakes.
- Application-data message-size limits (edge test E4) were not stress-tested. Host-side verification and decapsulation times were not measured.
- Test update images differ only in version. The PAS certificate slot (99 % full) leaves no room for an intermediate CA in an ML-DSA-87 chain.

10. Migration architecture: ECC-only to PQC-only

The brief excludes a hybrid ECC+PQC flow, so each plane is cut over *per device*. The fleet accepts both roots only during the transition window, and no single device runs a combined algorithm.

Step	Action	Production mechanism	Status on our board
1	Firmware update to a PQC-capable image	through the existing ECDSA-verified EPBL update path	shown (Tasks 1/2)
2	EPBL validation key ECDSA -> ML-DSA-87	BootROM-authenticated EPBL update (OEM ECDSA key)	shown by reflash; BootROM path documented
3	On-chip ML-DSA-87 identity + CSR, signed by a new ML-DSA-87 OEM CA	CSR delivered over a session authenticated by the still-valid ECC identity	keygen + CA shown; ECC bridge documented
4	Responder switches to the PQC-only SPDM profile (P4)	build-time profile, no classical algorithm offered	shown (Tasks 3/4)
5	Requesters drop and revoke the ECC root	fleet policy, CRL / anchor removal	classical root rejected - shown
6	Freeze PAS, close DAP, SECURE lifecycle, eFuse counter	irreversible	documented only (shared board)

11. Lessons learned, recommendations and business case

11.1 Lessons and engineering trade-offs

- **Verify the parameter set:** CNSA 2.0 means category 5 (ML-DSA-87, ML-KEM-1024) and SHA-384/512, not the commercial defaults or the documented SHA-256 pre-hash.
- **Bytes, not cycles:** PQC compute is fine on a Cortex-M33. Buffers, credential slots and the transport are where the design must change.
- **Defaults bite:** mismatched hash defaults between bootloader and signer produce images that silently never boot. Set both explicitly.
- **Attestation must be bound to verified boot:** placeholder measurements make an authenticated identity meaningless. We report EPBL-verified digests.
- **Test the negatives on the baseline too:** the same tamper campaign on ECC and PQC gives a fair comparison and avoids false confidence.
- **Trade-off taken:** one SPDM session and one certificate slot. Supporting several sessions or an intermediate CA would need more RAM or a larger PAS.

11.2 Recommendations for the next phase

- Production lock-down: OEM key + SECURE_APP policy for EPBL, PAS freeze, DAP closed, eFuse rollback counter.
- DICE-style device key derived from the measured image, binding the SPDM identity cryptographically to the boot chain.
- In-band firmware and credential update over SPDM/MCTP. Faster transport (I3C/PCIe VDM) to bring session time to seconds.
- Evaluate LMS for the bootloader (smaller verifier) and hardware acceleration of SHAKE/Keccak to cut ML-DSA signing variance.

11.3 Market and business viability

Every SPDM responder in GPUs, SmartNICs/DPUs, SSD controllers and CXL devices faces this migration. These devices have long service lives, and CNSA 2.0 sets dates for U.S. national-security suppliers, while CRA/NIS2 push lifecycle readiness in the EU. Our deliverable is a **reference migration kit on Infineon PSoC Control C3**: EPBL + SPDM integration, provisioning and test tooling, and a measured resource budget that lets a customer decide early. Adoption cost is low: the same responder code with a build-time profile and no new silicon for the PQC plane. The open item that does need silicon support is a PQC-capable immutable root.

12. Deliverables and reproduction

Brief deliverable	Where
1. Source code: EPBL integration, PSoC responder, host test application	epbl/bl (ML-DSA-87 config), patches/psoc-spdm-responder_task4.patch + BSP overlay, tools/resp_epbl_sign.py, tools/*.ps1, tools/demo_app
2. Build and run instructions (ModusToolbox)	README.md, results/HANDOFF.md, 'Reproduce' sections in the Task 1-4 reports, patches/README.md
3. ECC baseline and PQC configurations	feature_config_ECDSA-384_baseline.json / feature_config_ML-DSA-87.json; responder SPDM_PROFILE=P2 / P4
4. Credential generation and provisioning scripts	tools/task1_provision_p2.ps1, tools/task3_provision_p4.ps1, tools/task3_make_neg_roots.py

Brief deliverable	Where
5. Test report (boot + runtime, positive + negative)	sections 3-7; Task1-4 PDFs; run directories results/task*_*; tools/tests (offline suites)
6. Resource and performance comparison	section 8; results/TASK1..4_NOTES.md; tools/perf_summary.py, tools/epbl_ram_peak.ps1
7. Migration architecture	section 10; Task4_Lifecycle_Report.pdf
8. Five-minute live demonstration	results/Task5_Presentation.pdf, results/TASK5_DEMO.md, results/Demo_Presenter_Guide.pdf, tools/demo_app

Key reproduction commands (environment: `. tools\mtbenv.ps1`):

```
# boot plane: ML-DSA-87 EPBL + full test matrix
copy epbl\bl\platforms\PSC3_M6\feature_config_ML-DSA-87.json epbl\bl\platforms\PSC3_M6\feature_config.json
& $make -C epbl/bl build ; & $make -C epbl/bl qprogram
tools\task2_boot_tests.ps1
# SPDM plane: provision ML-DSA-87 identity, run requester suite and tamper campaign
tools\task3_provision_p4.ps1 ; tools\task1_spdm_runs.ps1 -Profile P4 ; tools\task3_neg_tests.ps1
# lifecycle: responder behind EPBL, update / rollback / tamper + SPDM measurements
tools\task4_epbl_spdm_tests.ps1 -Phase PREP ; tools\task4_epbl_spdm_tests.ps1 -Phase BOARD
# offline checks
py -3.13 -m unittest discover -s tools/tests -v
```

All work was done with Infineon tools and software as required. Reused: EPBL (CE43373), CE243041, SPDM CE (responder, requester, provisioning app/host), libspdm, wolfCrypt, mbedTLS. Built by the team: ML-DSA-87/SHA-512 boot configuration and signer change, the responder-behind-EPBL integration (BSP variant, merged signing, real measurements), PQC profile build and provisioning, fault-injection builds, test runners, measurement instrumentation, host verifier, report and demo tooling.